

PIVOTPIR: Compact Single-Server PIR via Rotated Key-Switching

Yue Chen
yuec12@illinois.edu

Ling Ren
renling@illinois.edu

Siebel School of Computing and Data Science
University of Illinois at Urbana-Champaign

Abstract

▷ **Pitch:** PIVOTPIR is a single-server PIR scheme with small requests. The core technique is the *rotated key-switching (RKS)* [Yue: Can change name], which significantly reduced the client-to-server communication by increasing the server computation slightly.

▷ **Two models:** PIVOT-RING (variant A): OnionPIRv2 variant with RKS; PIVOT-LWE (variant B): InsPIRe variant with RKS.

▷ **Update advantage in both models with good throughput:** Both models is naturally update-friendly. [Yue: Right now I am advertising the update-friendliness. Please decide if this is a good point to advertise.] Report numbers. Request in $\approx X$ KB / response overhead $\approx Y \times$ / throughput $\approx Z$ MB/s (stateful); total request $\approx XXX$ KB with no per-client state (double-stateless). Fill from results/.

1 Introduction

1.1 The single-server PIR problem and metrics

▷ **Definition:** A client retrieves entry i from an N -entry public database held by a single server, without revealing i to the server.

▷ **Single-server:** No non-collusion / multi-server assumption (one line contrasting with multi-server PIR).

▷ **Metrics:** (1) request size, (2) response overhead (response $\div$ entry size), (3) server throughput, (4) per-client server storage, (5) update efficiency.

▷ **What PIVOTPIR pushes on:** Small request and no per-client storage, while keeping throughput and updates competitive.

1.2 Common frameworks

▷ **The converged framework:** Modern SHE-based single-server PIR has converged to a common framework: arrange the database as a hypercube, then use homomorphic operations to select one slice of the hypercube at a time. The client compresses the query so the communication is small enough. (High-level story but no LWE/RLWE/SHE details?)

▷ **Stateful server:** Client uploads key material once; smallest request; server keeps per-client state.

- ▷ **Double-stateless:** No per-client server state; each request is self-contained and *unlinkable*; made practical by preprocessing.
 - ▷ **InsPIRe:** A recent state-of-the-art scheme that is double-stateless, but with large requests and heavy server precomputation. Not update-friendly.
 - ▷ **Our goal:** Push the double-stateless PIR to smaller requests and better update efficiency, while keeping throughput competitive.
-

1.3 Bottlenecks and our approach

- ▷ **The bottleneck for stateful server style schemes:** Communication is cheap, but requires large per-client server state. The key material is essential for query expansion. To expand one query to a large vector for first dimensional selection, the client needs to upload many expansion keys.
 - ▷ **The bottleneck for InsPIRe:** One database update invokes a large amount of server precomputation. We replace the crucial source of the precomputation (the polynomial evaluation) with more update-friendly operations (CMux gates with multiple selection bits).
 - ▷ **Both bottlenecks addressed:** The rotated key-switching (RKS) technique allows the client to upload only one or two expansion keys, and the server can simulate the rest of the expansion by rotating these keys. This reduces the amount of uploaded key material, while slightly increasing the server computation.
 - ▷ **Contributions:** (1) the RKS query-expansion engine and its key-material $\leftrightarrow$ compute trade-off; (2) a variant of OnionPIRv2 with RKS in the double-stateless model with the pseudurandom-side precompute and multi-root optimizations; (3) a variant of InsPIRe with RKS subsequent dimension after packing, which is more update-friendly; (4) an implementation and evaluation showing the metrics we care about, comparing with state-of-the-art schemes.
-

2 Preliminaries

2.1 Somewhat Homomorphic Encryption

Single-server PIR schemes built on lattice cryptography use leveled Fully Homomorphic Encryption, also called *Somewhat Homomorphic Encryption* (SHE), whose security relies on the hardness of Learning With Errors (LWE) or its ring variant (RLWE). We use two SHE schemes: an RLWE encryption scheme (instantiated with BFV [Bra12, FV12]) and RGSW [GSW13, CGGI20]. An SHE scheme supports a limited number of homomorphic additions and multiplications on ciphertexts. All known SHE schemes use *noisy* ciphertexts; homomorphic operations increase the noise, and once it grows too large the ciphertext no longer decrypts. Designing an efficient PIR scheme is, to a large extent, the art of keeping this noise growth under control, so we track it carefully for every operation we use.

RLWE encryption. We work over the fixed ring $R = \mathbb{Z}[x]/(x^n + 1)$ with a power-of-two n , ciphertext modulus q , and plaintext modulus t ; the secret key s is a polynomial with “small” coefficients drawn uniformly from $\chi_s = \{-1, 0, 1\}$, and a plaintext m lives in $R \bmod t$. An RLWE encryption of m under s is a pair of polynomials in $R \bmod q$, $\text{RLWE}_s(m) = (a, b) = (a, as + e + \Delta m)$ with $\Delta = \lfloor q/t \rfloor$, where a is uniform in $R \bmod q$, e is a small noise polynomial (coefficients from a discrete Gaussian χ_e), and Δ scales the message into the most significant bits. Decryption recovers m from the *phase* $\varphi_s(a, b) = b - as = \Delta m + e$ by rounding $\lfloor (b - as)/\Delta \rfloor \bmod t$, which is correct as long as $\|e\|_\infty < \Delta/2$, i.e. the noise does not exceed the gap Δ and pollute the message; the room left below this threshold is the *noise budget*. For convenience we also use a notation like RLWE but

without the scaling factor: $\text{RLWE}_s^*(m) = (a, as + e + m)$ encrypts m with no Δ , which is precise and convenient in the definitions below.

Gadget decomposition. Several operations below multiply by a ciphertext component without blowing up the noise; the tool is *gadget decomposition*, with base B and length ℓ . The signed decomposition $\mathbf{g}^{-1}(\cdot)$ takes a ring element x and writes every coefficient as ℓ digits in $[-B/2, B/2)$ against the gadget vector $\mathbf{g} = (B^{\ell-1}, \dots, B, 1)$, so that $\langle \mathbf{g}^{-1}(x), \mathbf{g} \rangle = x$; splitting into small digits is what turns multiplicative noise growth into additive. We also use *approximate* decomposition [CGGI16], which drops the lowest-order digits: it replaces $\mathbf{g}$ by a coarser gadget whose smallest entry is δ (so $\delta = 1$ recovers exact decomposition) and reconstructs x up to an error in $[-\delta/2, \delta/2)$. A smaller ℓ (larger δ) shrinks key material at the cost of a small extra error.

RLev. Stacking ℓ copies of RLWE^* that encrypt the same message scaled by the gadget entries gives an RLev ciphertext [WWL⁺24], $\text{RLev}_s(m) = (\text{RLWE}_s^*(mg_1), \dots, \text{RLWE}_s^*(mg_\ell))$. RLev carries no decryption algorithm. It appears only inside the homomorphic operations below and never needs to be decrypted.

2.2 Homomorphic Operations

Gadget product. The gadget product gives a clean way to describe the operations that follow. For a ring element u and an RLev ciphertext, it pairs the gadget decomposition of u against the RLev rows [KLD⁺23, WWL⁺24]:

$$u \odot \text{RLev}_s(m) := \langle \mathbf{g}^{-1}(u), \text{RLev}_s(m) \rangle = \sum_{i=1}^{\ell} \mathbf{g}^{-1}(u)_i \text{RLWE}_s^*(mg_i) \approx \text{RLWE}_s^*(um).$$

Because the digits $\mathbf{g}^{-1}(u)_i$ are small, the inner product between them and the RLev ciphertext incurs only additive noise growth (Appendix A).

Key switching. Key switching re-encrypts a ciphertext $c = (a, b) = \text{RLWE}_s(m)$ under a new key s' using a key-switching key $K_{s \rightarrow s'} = \text{RLev}_{s'}(s)$, an RLev encryption of s under s' [BV11]. It is one gadget product plus an addition:

$$\text{KS}(c, K_{s \rightarrow s'}) := (0, b) - a \odot K_{s \rightarrow s'} \approx \text{RLWE}_{s'}(m),$$

Ring automorphism and substitution. For an odd integer α , the map $\tau_\alpha : f(x) \mapsto f(x^\alpha)$ is a ring automorphism of R . The automorphisms are indexed by $\mathbb{Z}_{2n}^*$, the multiplicative group of odd residues modulo $2n$, and compose by index multiplication, $\tau_\alpha \circ \tau_\beta = \tau_{\alpha\beta \bmod 2n}$; the group is $\mathbb{Z}_{2n}^* \cong \mathbb{Z}_2 \times \mathbb{Z}_{n/2}$. Applied coefficient-wise to a ciphertext, τ_α permutes coefficients only — so it is *noise-free* — but it substitutes the secret, turning an encryption under $s(x)$ into one under $s(x^\alpha)$:

$$\tau_\alpha(\text{RLWE}_s(m)) = \tau_\alpha((a, b)) = (\tau_\alpha(a), \tau_\alpha(b)) = \text{RLWE}_{\tau_\alpha(s)}(\tau_\alpha(m)).$$

Composing τ_α with a key switch from $s(x^\alpha)$ back to $s(x)$ gives the *substitution* operation $\text{Subs}(\cdot, \alpha)$ [CCR19], which maps $\text{RLWE}_s(m(x))$ to $\text{RLWE}_s(m(x^\alpha))$ under a fixed key; its only cost is the one key switch, so its noise is again additive. The key switch takes the key-switching key $K_\alpha = \text{RLev}_s(\tau_\alpha(s))$, and each automorphism index α needs its own key.

3 Query Compression and Expansion via Rotated Key-Switching

[Yue: A short paragraph to motivate the need for compression. Consider merging the two paragraphs below into one. Just claim that we use a RLWE-level compression/expansion framework that is implicitly used in prior works.] With

the power of homomorphic encryption, the client can encode the selection vectors in ciphertexts, and the server easily evaluates linear combinations of the database entries homomorphically to output the desired encrypted result. However, the selection vector is usually sparse, containing many zero entries, resulting in a large communication overhead. Prior works [Yue: cite] compress these selection vectors into a single ciphertext, which the server expands back into the required selection vector. However, the expansion requires a large number of client’s secret-dependent ciphertexts, the key-switching keys, that are expensive to store and transmit. In the double-stateless model, where these key materials are not stored on the server side, the communication overhead of the key-switching keys dominates the total communication overhead. To address this issue, we propose a new technique called *rotated key-switching (RKS)*, which significantly reduces the communication overhead of the key-switching keys by using a small set of generating keys to derive all the required key-switching keys.

This section defines the compression and expansion procedures that both of our constructions share (Section 5 and Section 6). Both constructions need the server to hold a list of RLWE* ciphertexts, each encrypting a single constant. Sending them one at a time is wasteful, since a fresh ciphertext per constant is far larger than the constants themselves. Instead the client packs all the constants into the coefficients of one ciphertext (Section 3.1), and the server expands that ciphertext back into the list, so that the j -th output ciphertext encrypts the j -th packed constant (Section 3.2). The interface is simple: **Compress** takes a list of constants and returns one ciphertext; **Expand** takes that ciphertext and returns the list of per-constant RLWE* ciphertexts. How each construction uses these constants is deferred to Section 5 and Section 6.

The common reference string. [Yue: Shall we put this somewhere else? Maybe in the introduction or preliminary.] In every RLWE ciphertext (a, b) the polynomial a is uniformly random and independent of the message and the secret. Both parties can therefore derive a from a shared public seed, and the client transmits only b , which depends on both the message and the client’s secret key. The seed acts as a common reference string (CRS). The same convention applies to every ciphertext that the client uploads, which reduces the communication size by half, and it lets the server precompute on the a polynomials before any query arrives. In the CRS model, the CRS remains secure as long as the client generates a fresh secret key for each query. Reusing one secret against the fixed CRS would let the server recognize when two queries are related. Prior works [MPSY26, HHCG⁺23], [Yue: hintless,YPIR] also work in the smae model.

3.1 The Compressed Query

The compression algorithm (Algorithm 1) is relatively standard and appears in many prior works [Yue: cite]. We present it here for completeness and to establish notation.

Given w constants $c_0, \dots, c_{w-1} \in \mathbb{Z}_q$, Algorithm 1 places them in the coefficients of one polynomial and encrypts it, outputting a single ciphertext together with a small set of key-switching keys. The key-switching keys are $K_{\alpha_1}, \dots, K_{\alpha_G}$ for a small set of automorphism indices $\{\alpha_1, \dots, \alpha_G\} \subset \mathbb{Z}_{2n}^*$, the *generating set*; Section 3.3 explains how it is chosen and why a small set suffices. These materials are everything the server needs to expand the ciphertext back into the w per-constant ciphertexts. They follow the CRS convention above: the client transmits only the b polynomials, which depend on the secret key and the constants.

Remarks. Line 3 of Algorithm 1 divides the polynomial by 2^h . Every expansion level doubles the underlying coefficients, so after h levels each constant returns to its original scale. See [CCR19] for a detailed discussion.

Algorithm 1: Compress (client)

Notation: τ_α the automorphism $x \mapsto x^\alpha$, indexed by $\mathbb{Z}_{2n}^*$; the polynomial a and the random half of every key row come from the CRS.

Input: secret key s ; constants $c_0, \dots, c_{w-1} \in \mathbb{Z}_q$; generating set $\{\alpha_1, \dots, \alpha_G\} \subset \mathbb{Z}_{2n}^*$.

Output: request $\text{req} = (\tilde{b}, \{K_{\alpha_1}, \dots, K_{\alpha_G}\})$: the transmitted half $\tilde{b}$ of the compressed ciphertext and the generator key-switching keys.

```
1  $h \leftarrow \lceil \log_2 w \rceil$ 
2  $f \leftarrow \sum_{j=0}^{w-1} c_j x^j$ 
3  $f \leftarrow 2^{-h} f \bmod q$ 
4  $\tilde{b} \leftarrow a s + e + f$ 
5  $K_{\alpha_j} \leftarrow \text{RLev}_s(\tau_{\alpha_j}(s)), \quad \text{for } j = 1, \dots, G$ 
6 return  $(\tilde{b}, \{K_{\alpha_j}\}_{j=1}^G)$ 
```

3.2 The Expansion Tree

The expansion (Algorithm 2) runs in h levels and forms a binary tree. The root is the compressed ciphertext. At every level each ciphertext splits into two, so after all h levels there are 2^h leaves, each an RLWE^{*} ciphertext encrypting one packed coefficient as its constant term. [\[Yue: figure: expansion tree\]](#)

The split in line 4 uses one automorphism per level. Take the first level as an example: the automorphism τ_{n+1} sends x to $x^{n+1} = -x$, which negates every odd-degree coefficient and fixes every even-degree coefficient. Adding the image back to the original ciphertext cancels the odd coefficients and doubles the even ones. Similarly, subtracting the image from the original keeps the odd coefficients and cancels the even ones. The next level repeats the process but on sparser underlying polynomials. In general, level r must separate coefficients whose indices differ by 2^r , and the automorphism τ_{η_r} with $\eta_r = n/2^r + 1$ exactly negates and separates those coefficients. This is why each level needs a different automorphism, and therefore its own key-switching key. Since one substitution costs one key switch, the whole tree costs one key-switching operation per internal node.

Remarks. Two details are standard and we keep them out of the pseudocode. First, the expansion returns the packed constants up to a fixed permutation: the output order is determined by the order of the two children in line 4 of Algorithm 2 (it is the bit-reversal of the packing order), and the client simply places its constants so that they come out in the intended positions. Second, w need not be a power of two: the server skips every subtree that contains no useful leaf, so the work scales with w rather than 2^h . See [\[Yue: OnionPIRv2\]](#) for both details.

3.3 Rotated Key-Switching (RKS)

The expansion tree applies the substitution $\text{Subs}(\cdot, \eta_r)$ at each level r . Realized directly, each of the h distinct level automorphisms needs its own key-switching key, so the client uploads h keys, and these keys dominate the request size. We remove this coupling with *rotated key-switching* (RKS), generalizing the single-generator construction of WHISPIR [dCLS24].

Recall from Section 2 that automorphism indices compose by multiplication: $\tau_\alpha \circ \tau_\beta = \tau_{\alpha\beta \bmod 2n}$. The idea is to upload keys for only a few *generators* and reach every level automorphism by a *chain* of substitutions. For simplicity, consider two generators $\alpha, \beta \in \mathbb{Z}_{2n}^*$. The client uploads the two keys K_α and K_β . To reach a level index that factors as $\eta_r = \alpha^i \beta^j \bmod 2n$, the server applies i

Algorithm 2: Expand (server)

Notation: $\eta_r = n/2^r + 1$, the automorphism index of expansion level r ; $\text{Subs}(\cdot, \eta)$ the substitution of Section 2; $\mathbf{L}$ is the list of ciphertexts at the current expansion level.

Input: request $(\tilde{b}, \{K_{\alpha_j}\}_{j=1}^G)$; the number of constants w .

Output: RLWE^{*} ciphertexts $(C_0, \dots, C_{w-1})$, where C_j encrypts the packed constant c_j in its constant term.

- 1 $\mathbf{L} \leftarrow ((a, \tilde{b}))$ with a regenerated from the CRS; $h \leftarrow \lceil \log_2 w \rceil$
 - 2 **for** $r \leftarrow 0$ **to** $h - 1$ **do**
 - 3 derive the key for τ_{η_r} from $\{K_{\alpha_j}\}$ by rotated key-switching (Section 3.3)
 - 4 $\mathbf{L} \leftarrow (c + \text{Subs}(c, \eta_r), x^{-2^r}(c - \text{Subs}(c, \eta_r)) : c \in \mathbf{L})$
 - 5 **return** $(C_0, \dots, C_{w-1}) \leftarrow$ the first w entries of $\mathbf{L}$
-

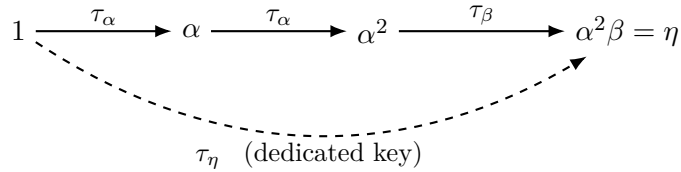


Figure 1: An example of RKS reaching a level automorphism τ_η with $\eta = \alpha^2\beta$. The chain (top) applies the base-generator substitutions $\tau_\alpha, \tau_\alpha, \tau_\beta$, reusing only the keys of the two generators; the direct route (bottom, dashed) is a single substitution with a key dedicated to τ_η . Both compose to the same automorphism, so RKS trades extra substitutions for fewer uploaded keys.

substitutions with K_α and j with K_β (Figure 1). The chain computes exactly τ_{η_r} ; the only price is that each chain step adds one key switch’s noise. For the tree heights we use, a single well-chosen generator already reaches every level (the WHISPIR case); a second generator shortens the chains, reducing both the substitution count and the accumulated noise. RKS generalizes to any number of generators G , and two is already a good balance for PIR.

Rotating ciphertexts or rotating keys. A chain step can be applied in two equivalent ways. The first rotates the running ciphertext: apply τ_α to the ciphertext, then key-switch with K_α . The second rotates the stored key instead [dCLS24]: applying τ_β to every row of K_α turns it into the key that switches $\tau_{\beta\alpha}(s)$ to $\tau_\beta(s)$, so the server can assemble, once, the exact key a level needs, and then use it on every ciphertext of that level with a single key switch each. Rotating a key costs more than rotating one ciphertext, because a key holds ℓ_{KS} ciphertexts, but the rotated key is reused across the whole level. We therefore use a hybrid rule. In the shallow levels, which hold few ciphertexts, we rotate the ciphertexts. In the deep levels, which hold many, we rotate the key once and key-switch every ciphertext directly. Both ways add only the additive key-switch noise; the rotation itself is a noise-free permutation.

Relaxing the level targets. The substitution at level r need not equal τ_{η_r} on the whole ring. It only has to act correctly on the coefficients that are still present. The expanded message grows sparser with depth, as mentioned in Section 3.2. The input of level r has nonzero coefficients only at exponents divisible by 2^r . On such an exponent the automorphism is insensitive to a large part of its index, which enlarges the set of admissible targets.

Lemma 1. *Fix a level r and let $\eta \in \mathbb{Z}_{2n}^*$ satisfy $\eta \equiv \eta_r \pmod{2n/2^r}$. Then τ_η and τ_{η_r} send every monomial x^k with $2^r \mid k$ to the same element of R . Replacing τ_{η_r} by any such τ_η therefore leaves*

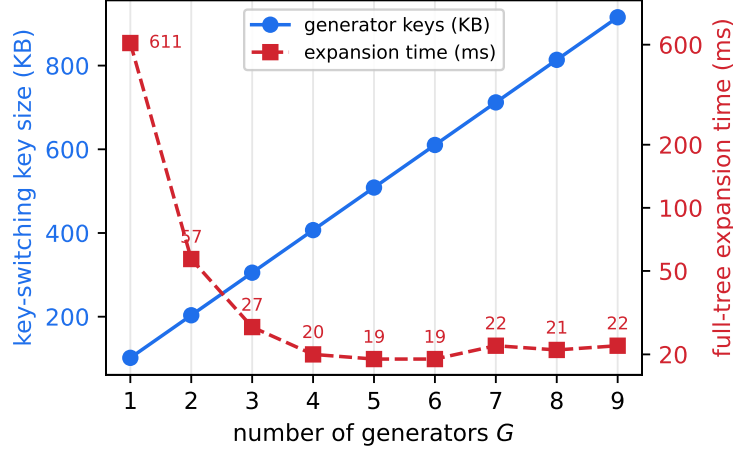


Figure 2: Key material against server expansion time as the number of generators G varies, for a full expansion tree of height $h = 9$ (512 leaves, $n = 2048$, key-switching gadget fixed at $\ell_{KS} = 7$, $B_{KS} = 2^8$). The key size is exact and grows linearly in G ; the expansion time (log scale) collapses once the chains shorten, and levels off around $G = 4$.

the output of expansion level r unchanged.

Proof. Write $k = 2^r m$. From $\eta \equiv \eta_r \pmod{2n/2^r}$ we have $\eta - \eta_r = t(2n/2^r)$ for some integer t , so $(\eta - \eta_r)k = t(2n/2^r)2^r m = 2ntm \equiv 0 \pmod{2n}$. Hence $x^{\eta k} = x^{\eta_r k}$ in $R = \mathbb{Z}[x]/(x^n + 1)$, where $x^{2n} = 1$. Level r acts only on such monomials, so its output does not change. $\square$

The admissible indices for level r thus form the coset $\eta_r + \frac{2n}{2^r}\mathbb{Z}$ in $\mathbb{Z}_{2n}^*$, which has 2^r elements. The size doubles at every level and is largest deep in the tree. The generator search then only needs to reach *some* admissible index at each level rather than the single index η_r , which shortens the chains. With one generator ($G = 1$) and a tree of height $h \leq \log_2 n - 1$, the single index [Yue: proposed by Hyesung Ji] $\alpha = 1 + n/2^{h-1}$ reaches an admissible target at every level, giving a closed-form generating set.

Generator selection. [Yue: a short offline search over $\mathbb{Z}_{2n}^*$ minimizes total chain length across all tree levels; the search algorithm is NP-hard. Our solution relies on heuristics not shown in the paper, but the algorithm lives in our codebase. We can provide a table of best generators we find and the number of key-switchings required for expanding the whole tree. This algorithm takes in: G the number of generators, h the height of the tree, n the polynomial degree; it then outputs a generating set and the total number of key switchings. Fix h and n , make G the only variable.]

Trade-offs. See Figure 2. Key material $\propto G \cdot \ell_{KS}$ many polynomials. Two extremes: $G = h$ (max keys and min compute) and $G = 1$ (min keys / longest chains). PIVOTPIR sits in between and select a balanced option via a short offline search.

4 Homomorphic Operations Used by the PIR

Beyond the operations in Section 2.2, both constructions fold hypercube dimensions with a selection primitive built from a second ciphertext type.

RGSW. An RGSW ciphertext assembles two correlated RLev halves, one masked by the secret, $\text{RGSW}_s(m) = (\text{RLev}_s(-sm), \text{RLev}_s(m))$, so it consists of 2ℓ RLWE^{*} rows.

External product. The external product multiplies an RLWE ciphertext $c = (a, b) = \text{RLWE}_s(m_1)$ by an RGSW ciphertext $\text{RGSW}_s(m_2) = (\text{RLev}_s(-sm_2), \text{RLev}_s(m_2))$ under the same key, using two gadget products:

$$c \boxtimes \text{RGSW}_s(m_2) := a \odot \text{RLev}_s(-sm_2) + b \odot \text{RLev}_s(m_2) \approx \text{RLWE}_s(m_1 m_2),$$

at a cost of 4ℓ polynomial multiplications. Its noise is additive: two gadget terms plus the input noise scaled only by the (small) RGSW message m_2 [CGGI16]. In this work m_2 is either a selection bit or the small secret-key polynomial (low L_2 norm), so a chain of external products keeps the noise low — this asymmetry is why we fold dimensions with external products rather than BFV multiplication.

CMux. CMux from [CGGI20] takes two RLWE ciphertexts and one RGSW selection bit, and returns one of the two.

RLWE-to-RGSW conversion. The expansion of Section 3 delivers selection bits as RLWE^* ciphertexts, but the fold needs them as RGSW. Given the ℓ ciphertexts $c_j = (a_j, b_j) = \text{RLWE}_s^*(m g_j)$ — together an $\text{RLev}_s(m)$ — and the conversion key $K_{\text{conv}} = \text{RLev}_s(s^2)$, one gadget product per row builds the missing masked half [WWL⁺24]:

$$\text{ToRGSW}((c_1, \dots, c_\ell), K_{\text{conv}}) : \quad \hat{c}_j \leftarrow (b_j, 0) + a_j \odot K_{\text{conv}}, \quad j = 1, \dots, \ell.$$

The trivial ciphertext $(b_j, 0)$ has phase $-b_j s$ and the gadget product contributes phase $a_j s^2$, so $\hat{c}_j$ has phase $-s(b_j - a_j s)$, an RLWE^* encryption of $-s m g_j$. The rows $(\hat{c}_1, \dots, \hat{c}_\ell)$ therefore form $\text{RLev}_s(-sm)$, and together with the input rows they assemble $\text{RGSW}_s(m) = (\text{RLev}_s(-sm), \text{RLev}_s(m))$. The cost is ℓ gadget products and the noise is additive.

5 PIVOT-RING: Double-Stateless PIR

5.1 Database Layout

▷ **Hypercube:** DB of N entries as dimensions $N_0 \times 2 \times \dots \times 2$ (d size-2 dims). Justify: first dim large because it is answered by the cheap BFV plaintext step; rest size-2 using the CMux.

▷ **First dimension:** plaintext (DB) $\times$ ciphertext (query) matrix product; state the cost model (memory-bandwidth bound, this is the throughput number) and the noise (one pt-ct mult, multiplicative but applied *once*).

▷ **Subsequent dimensions:** each size-2 dim folded by an external product against an RGSW selection bit; noise additive across the d folding dims.

5.2 The Protocol

6 PIVOT-LWE: The INSPIRE Variant

▷ **Pipeline overview (with figure):** client sends a b -parts-only LWE one-hot query for the first dimension (the A matrix is public and ring-structured) plus one compressed selector query $\rightarrow$ server: SIMPLEPIR first dimension $\rightarrow$ InspiRING packing of the output LWEs into RLWE $\rightarrow$ Expand $\rightarrow$ CMux folding $\rightarrow$ modulus switching and response.

Algorithm 3: PIVOT-RING: one query

Notation: database $D = (D_{k,j}) \in R_t^{N_0 \times 2^d}$, one plaintext polynomial per cell; $\Delta = \lfloor q/t \rfloor$;
 $\mathbf{g} = (g_1, \dots, g_\ell)$ the gadget vector.

Input: target coordinates $(i_0, i_1, \dots, i_d)$ with $i_0 \in \{0, \dots, N_0 - 1\}$ and $i_1, \dots, i_d \in \{0, 1\}$;
generating set $\{\alpha_1, \dots, \alpha_G\}$.

Output: the client learns D_{i_0, j^*} , where $j^* = \sum_{r=1}^d i_r 2^{d-r}$.

- 1 **Client:** sample a fresh secret s
 - 2 $w \leftarrow N_0 + \ell d$; $(c_0, \dots, c_{w-1}) \leftarrow$ all zeros, then $c_{i_0} \leftarrow \Delta$ and $c_{N_0+(r-1)\ell+(j-1)} \leftarrow i_r g_j$ for
 $r = 1, \dots, d, j = 1, \dots, \ell$ // **one-hot, then the gadget-scaled bits**
 - 3 $\text{req} \leftarrow \text{Compress}(s, (c_0, \dots, c_{w-1}), \{\alpha_1, \dots, \alpha_G\})$; $K_{\text{conv}} \leftarrow \text{RLev}_s(s^2)$; send
 $(\text{req}, K_{\text{conv}})$
 - 4 **Server:** $(C_0, \dots, C_{w-1}) \leftarrow \text{Expand}(\text{req}, w)$
 - 5 $y_j \leftarrow \sum_{k=0}^{N_0-1} D_{k,j} \cdot C_k$, for $j = 0, \dots, 2^d - 1$ // **first dimension, PtCtMul**
 - 6 $S_r \leftarrow \text{ToRGSW}((C_{N_0+(r-1)\ell}, \dots, C_{N_0+r\ell-1}), K_{\text{conv}})$, for $r = 1, \dots, d$
 - 7 **for** $r \leftarrow 1$ **to** d **do**
 - 8 $\lfloor m \leftarrow 2^{d-r}$; $y_j \leftarrow \text{CMux}(S_r, y_{j+m}, y_j)$, for $j = 0, \dots, m - 1$
 - 9 send $z \leftarrow \text{ModSwitch}(y_0)$ to the client
 - 10 **Client:** output $\text{Dec}_s(z)$
-

▷ **Update-friendliness:** show the update problem in the original INSPIRE pipeline: a single update in the raw database triggers a large amount of updates in the preprocessed database (figure highlighting the affected data). In PIVOT-LWE a single raw-data update touches one hint row and one packing batch.

▷ **Correctness and privacy:** follows INSPIRE and Theorems 1–4.

7 Implementation Techniques

7.1 Dense Plaintext Storage (Squishing)

▷ **What:** store several plaintext values per machine word for the first-dimension scan: three 10-bit values per u32, or five 12-bit values per u64. Borrowed from SIMPLEPIR’s Basis/Squishing storage. The scan is memory-bandwidth-bound, so scan bytes translate directly into throughput.

▷ **Numbers:** storage overhead relative to raw data (word padding $\times$ slot inflation); measured effect in eval.

7.2 A Memory-Bound First Dimension

▷ **What:** the first-dimension product runs directly at the ciphertext modulus with lazy-reduction kernels: residues accumulate raw in machine words with rare reductions, each loaded piece of the query is reused across a tile of database rows, and on IFMA-capable CPUs each multiply-add pair is a single fused 52-bit multiply-add instruction (products fit its exact window). The scan reaches the machine’s measured single-core memory bandwidth up to the cost of unpacking the dense storage; the hint $D \cdot A$ is precomputed at NTT speed thanks to the ring structure of A (the YPIR silent-preprocessing idea).

Algorithm 4: PIVOT-LWE: one query

Notation: database $M \in \mathbb{Z}_t^{\gamma 2^d \times N_0}$, one entry per cell, rows grouped into 2^d blocks of γ rows; $A \in \mathbb{Z}_q^{N_0 \times n}$ the public LWE matrix, derived from the CRS; $\vec{s} \in \mathbb{Z}^n$ the coefficient vector of s ; $\mathbf{g} = (g_1, \dots, g_\ell)$ the gadget vector; **Pack** the LWE-to-RLWE packing of INSPIRE, using the packing key K_{pack} .

Input: target coordinates: column $i_0 \in \{0, \dots, N_0 - 1\}$; block bits $i_1, \dots, i_d \in \{0, 1\}$; row $\rho \in \{0, \dots, \gamma - 1\}$ inside the block.

Output: the client learns $M[\gamma j^* + \rho, i_0]$, where $j^* = \sum_{r=1}^d i_r 2^{d-r}$.

- 1 **Server, offline (per database):** hint $H \leftarrow MA \bmod q$
 - 2 **Client:** sample a fresh secret s ; $\vec{q} \leftarrow A\vec{s} + \vec{e} + \Delta u_{i_0} \bmod q$, with $u_{i_0} \in \{0, 1\}^{N_0}$ the one-hot vector
 - 3 $w \leftarrow \ell d$; $c_{(r-1)\ell+(j-1)} \leftarrow i_r g_j$, for $r = 1, \dots, d$ and $j = 1, \dots, \ell$;
req $\leftarrow \text{Compress}(s, (c_0, \dots, c_{w-1}), \{\alpha_1, \dots, \alpha_G\})$; $K_{\text{conv}} \leftarrow \text{RLev}_s(s^2)$; generate K_{pack} for s ; send $(\vec{q}, \text{req}, K_{\text{conv}}, K_{\text{pack}})$
 - 4 **Server, online:** $\vec{v} \leftarrow M\vec{q} \bmod q$ // row k of $(H, \vec{v})$ is an LWE ciphertext of $\Delta M[k, i_0]$
 - 5 $p_j \leftarrow \text{Pack}((H_k, v_k)_{k \in \text{block } j}, K_{\text{pack}})$, for $j = 0, \dots, 2^d - 1$
 - 6 $(C_0, \dots, C_{w-1}) \leftarrow \text{Expand}(\text{req}, w)$
 - 7 $S_r \leftarrow \text{ToRGSW}((C_{(r-1)\ell}, \dots, C_{r\ell-1}), K_{\text{conv}})$, for $r = 1, \dots, d$
 - 8 **for** $r \leftarrow 1$ **to** d **do**
 - 9 $m \leftarrow 2^{d-r}$; $p_j \leftarrow \text{CMux}(S_r, p_{j+m}, p_j)$, for $j = 0, \dots, m - 1$
 - 10 send $z \leftarrow \text{ModSwitch}(p_0)$ to the client
 - 11 **Client:** decrypt z and output coefficient ρ
-

▷ **Numbers:** matvec vs the same node's STREAM bandwidth; the decomposition read / +unpack / +multiply.

7.3 Public-Seed a -Parts Everywhere

▷ **What:** every public ciphertext's a -part (query roots, generator keys, packing keys, first-dimension A) derives from public seeds; only b -parts travel, and every data-independent half of the server work (expansion tree a -side, packing digit tables, first-dimension hint) is precomputed once per database and shared across all clients and queries. This split is what the Compress/Expand interface of Section 3 is built on.

7.4 Approximate Gadget Decomposition and Base Selection

▷ **What:** approximate (bit-dropping) decomposition for the packing, key-switching and folding gadgets; with a single packing digit ($\ell_{\text{pack}} = 1$) the base is a free parameter and is set at the analytic balance point between the gadget term and the rounding term. Same balance argument selects the folding gadget base under the constraint that the number of selector rows keeps the expansion tree height unchanged.

▷ **Numbers:** the measured base sweep (plateau + cliffs), and the selector-gadget retune that the 12-bit plaintext modulus required.

7.5 Response Modulus Switching

▷ **What:** the final RLWE response is switched to a small NTT-friendly modulus before shipping; standard, but worth stating since it sets the download size.

8 Evaluation

▷ **Setup paragraph:** hardware (Xeon Platinum 8358, Ice Lake, AVX-512), single physical core, hyper-threading off, single thread; HEXL for NTT; methodology note (first dim is bandwidth-bound). State which numbers are hardware-independent (sizes, generator sets, k_{skip} , noise budgets — reproduce exactly) vs. hardware-dependent (timings).

8.1 Main Comparison

▷ **Table 1: end-to-end efficiency:** PIVOTPIR (both models) vs. ONIONPIR, SPIRAL, FSPIR, KSPIR, RESPIRE, WHISPIR, INSPIRE at matched DB size / entry size. Columns: request split to query + key materials size, response, overhead (= resp/entry), throughput.

▷ **Table 2: key size vs expansion time:** sweep number of generators G (equivalently, expansion policy) and report: request size, expansion time, and post-expansion noise budget. Show the key-material $\leftrightarrow$ compute Pareto curve and mark the two extremes ($G = h$, $G = 1$) and the chosen operating point. Find a way to show the noise budget

▷ **DB update efficiency:** measure the cost of the DB update in both models, compare them to INSPIRE's per-update work.

8.2 The p - ℓ Tradeoff, Isolated

▷ **The claim:** pack cost per slot is proportional to ℓ , raw data per slot is $\lfloor \log_2 p \rfloor$ bits, and the packing noise is an ABSOLUTE quantity (gadget + rounding terms, independent of p). So shrinking p only buys decryption headroom, and the right operating point is the largest p that still packs with a single approximate digit.

▷ **The experiment (test pack_pl_sweep in our codebase):** pack n LWE encryptions of zero once per gadget length ($\ell = 1..4$, base tuned per ℓ); with $m = 0$ the packed phase IS the pack noise, so one measurement per ℓ determines feasibility for EVERY p analytically: $\text{budget}(p) = \log_2((q/2p)/\text{noise}_\ell)$. Output: the staircase $\ell_{\min}(\log p)$ and the cost-per-raw-bit curve — falls within each ℓ -plateau, jumps at each transition, minimum at the largest $\ell = 1$ -feasible width. The test machine-checks this.

▷ **Figure/table:** measured at $q = 2^{58}$, $n = 2048$: noise $2^{41.1}/2^{32.1}/2^{27.5}/2^{25.4}$ for $\ell = 1..4$; time 8/16/22/28 ms per 2048 slots (linear in ℓ); $\ell = 1$ feasible to width 13; the 2^{16} point needs $\ell = 2$ (with a balanced approximate base — one digit fewer than INSPIRE's exact gadget) and is still $1.47\times$ worse per raw bit than width 12 at $\ell = 1$.

▷ **Pipeline cap:** in the full pipeline the CMux-fold noise floor caps $\log p \lesssim 13$ anyway, so the packing optimum and the pipeline constraint agree.

8.3 Case Study: Smaller p with $\ell = 1$ Packing, Inside INSPIRE

-
- ▷ **Why this experiment:** it demonstrates the design logic behind PIVOT-LWE’s parameters: reducing the plaintext modulus p and the packing gadget length ℓ TOGETHER is beneficial — the smaller p buys the noise headroom that a single approximate packing digit needs, and the single digit makes packing (the bottleneck) proportionally cheaper with $3\times$ smaller packing keys. Run in INSPIRE’s own codebase, against their own baseline, with their own correctness assert, so the comparison is on their terms.
 - ▷ **Table: baseline vs. p 10-bit vs. p 12-bit:** columns: parameters (p , ℓ , storage form), first-dimension time, packing time, online total, upload keys, request. Two storage forms per small- p point: as-is (INSPIRE’s u16-padded storage) and densely packed (the squishing of Section 7) — the padding artifact is exactly what initially hid the win.
 - ▷ **Interp caveat:** at 12-bit p the headroom left for INSPIRE’s polynomial-evaluation degree shrinks (each extra bit of p costs $\approx 4\times$ in their maximum interpolation degree); report the operating point honestly (the packing/scan wins do not depend on it; the download size does).
-

9 Related Work

Keep it a genuine related-work section (comparison of ideas), not a “we beat X” list — consistent with the clean-slate framing.

- ▷ **SHE-based single-server PIR lineage:** SEALPIR [ACLS18] (query compression / packing), ONION-PIR [MCR21] (external-product folding replaces noisy mults), SPIRAL [MW22] (matrix Regev / gadget encoding), RESPIRE [BMW24], WHISPIR [dCLS24] (rotate-the-key idea we build the RKS chain on), INSPIRE [MPSY26] (CRS model / stateless comparison point). For each: one sentence on the idea we borrow or contrast.
 - ▷ **Preprocessing / stateful PIR:** SIMPLEPIR and the hint-based line — contrast their client-hint state with our two models’ state choices.
 - ▷ **Multi-server / IT-PIR:** one paragraph, out of scope, cite for completeness ([CKGS98, Ste98]).
 - ▷ **Position:** close by placing PIVOTPIR in the space: same converged skeleton, new expansion engine (RKS), and the first to offer one construction across stateful and double-stateless with a clean update story.
 - ▷ **Placement decision:** could move before eval or fold into intro depending on venue; leave as its own section for now.
-

10 Conclusion

-
- ▷ **Recap:** PIVOTPIR: a compact single-server PIR whose RKS expansion engine decouples key material from expansion depth, instantiated in a stateful and a double-stateless model from one construction.
 - ▷ **Takeaway:** the key-material $\leftrightarrow$ compute trade-off is a knob, not a fixed cost; and statelessness is affordable once expansion keys are cheap.
 - ▷ **Future work:**
-

References

- [ACLS18] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and*

- Privacy*, pages 962–979, San Francisco, California, USA, 2018. IEEE Computer Society.
- [BMW24] Alexander Burton, Samir Jordan Menon, and David J Wu. Respire: High-rate pir for databases with small records. *Cryptology ePrint Archive*, 2024.
 - [Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer, 2012.
 - [BV11] Zvika Brakerski and Vinod Vaikuntanathan. Fully Homomorphic Encryption from Ring-LWE and Security for Key Dependent Messages. In *Advances in Cryptology – CRYPTO 2011*, pages 505–524. Springer, Berlin, Germany, 2011.
 - [CCR19] Hao Chen, Ilaria Chillotti, and Ling Ren. Onion ring ORAM: efficient constant bandwidth oblivious RAM from (leveled) TFHE. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, CCS*, pages 345–360, London, UK, 2019. ACM.
 - [CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *Advances in Cryptology - ASIACRYPT- 22nd International Conference on the Theory and Application of Cryptology and Information Security*, pages 3–33, Hanoi, Vietnam, 2016. eprint.
 - [CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.*, 33(1):34–91, 2020.
 - [CKGS98] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *J. ACM*, 45(6):965–981, 1998.
 - [dCLS24] Leo de Castro, Kevin Lewi, and Edward Suh. Whispir: Stateless private information retrieval with low communication. *Cryptology ePrint Archive*, 2024.
 - [FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.*, 2012:144, 2012.
 - [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference*, pages 75–92, Santa Barbara, CA, USA, 2013. Springer.
 - [HHCG⁺23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One Server for the Price of Two: Simple and Fast {Single-Server} Private Information Retrieval, 2023. [Online; accessed 5. May 2026].
 - [KLD⁺23] Andrey Kim, Yongwoo Lee, Maxim Deryabin, Jieun Eom, and Rakyong Choi. LFHE: Fully homomorphic encryption with bootstrapping key size less than a megabyte. *Cryptology ePrint Archive*, Paper 2023/767, 2023.
 - [MCR21] Muhammad Haris Mughees, Hao Chen, and Ling Ren. Onionpir: Response efficient single-server pir. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, pages 2292–2306, 2021.

- [MPSY26] Rasoul Akhavan Mahdavi, Sarvar Patel, Joon Young Seo, and Kevin Yeo. InsPIRe: Communication-Efficient PIR with Server-side Preprocessing. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 1240–1259. IEEE Computer Society, May 2026.
- [MW22] Samir Jordan Menon and David J Wu. Spiral: Fast, high-rate single-server pir via fhe composition. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 930–947. IEEE, 2022.
- [Ste98] Julien P. Stern. A new efficient all-or-nothing disclosure of secrets protocol. In *Advances in Cryptology - ASIACRYPT '98, International Conference on the Theory and Applications of Cryptology and Information Security*, volume 1514 of *Lecture Notes in Computer Science*, pages 357–371, Beijing, China, 1998. Springer.
- [WWL⁺24] Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit Bootstrapping: Faster and Smaller. In *Advances in Cryptology – EUROCRYPT 2024*, pages 342–372. Springer, Cham, Switzerland, May 2024.

A Precise Noise Analysis

[Yue: I will deal with this section later]